

Section 3, Problem 2

Food Ordering System using gRPC

Distributed Systems — Homework 3

1 What the question asks

Build a simplified food ordering service, in the manner of Swiggy, using gRPC. A central server holds a fixed set of restaurants and the state of every order. Multiple customer clients place and track orders; multiple restaurant clients receive orders and advance their status. The server coordinates between them.

Section 3 of the assignment offers two problems — Collaborative Document Editing and this one. This submission implements **Problem 2**.

Five RPCs are required: `ListRestaurants`, `PlaceOrder`, `GetOrderStatus`, `UpdateOrderStatus`, and `SubscribeToOrderUpdates`, the last of which must be a server-streaming call so that customers receive status changes as they happen.

2 What was built

File	Role	Lines
<code>food_ordering.proto</code>	the service definition and its messages	60
<code>server.py</code>	restaurants, orders, state validation, streaming, locking	170
<code>customer.py</code>	the customer CLI: list, order, check, track, cancel	111
<code>restaurant.py</code>	the restaurant CLI: view, accept, prepare, mark ready	64

The server address is a command-line argument, as specified: `python server.py localhost:50051`. Both clients take the server address the same way, and the restaurant client additionally takes its own name, so it sees only its own orders.

2.1 Order state

PLACED → ACCEPTED → PREPARING → READY, PLACED → CANCELLED

The permitted transitions are held in a single table, and any request not matching one is rejected. Keeping the rule in one place rather than scattering `if` statements through the handlers is what makes it possible to say with confidence that, for instance, a `READY` order can never return to `PREPARING`. Cancellation is permitted only from `PLACED`: once a restaurant has accepted an order, cancelling it is refused.

2.2 Concurrency

All shared state — the order table and the subscriber lists — is protected by a single lock. A single lock is the right choice here because the critical sections are tiny (a dictionary lookup and

an assignment) and the alternative, one lock per order, would add a way to deadlock in exchange for contention that does not exist at this scale.

Notifications are dispatched *outside* the lock. Each subscriber has its own queue; the handler copies the subscriber list while holding the lock, releases it, and only then puts the update on each queue. If notification happened inside the lock, one slow or unresponsive subscriber would block every other request to the server.

2.3 Exception handling

All six situations named in the assignment return a gRPC status code rather than an exception or a silent failure.

Situation	Status code
Order from a non-existent restaurant	NOT_FOUND
Order an unavailable food item	NOT_FOUND
Access a non-existent order	NOT_FOUND
Restaurant updates another restaurant's order	PERMISSION_DENIED
Invalid status transition	FAILED_PRECONDITION
Cancel an order already accepted	FAILED_PRECONDITION

The distinction between the codes carries meaning: `NOT_FOUND` says the thing does not exist, `PERMISSION_DENIED` says it exists but is not yours, and `FAILED_PRECONDITION` says it exists and is yours but the system is in the wrong state for what you asked. A client can act differently on each.

3 Verification

Each requirement was exercised against a running server rather than reasoned about.

- `ListRestaurants` returns both restaurants with the menus given in the assignment.
- An order for two Margherita Pizzas returns a unique identifier, the correct total of 500, and status `PLACED`.
- The full cycle `ACCEPTED` → `PREPARING` → `READY` succeeds, and each change is delivered to a subscribed customer over the stream, in order.
- All six error conditions above return their intended status code.
- Twenty orders placed simultaneously from twenty threads produced twenty distinct identifiers and no errors.

4 A defect found by measurement, and its repair

Functional testing passed every case above, and the system still had a serious fault.

`SubscribeToOrderUpdates` originally waited on its queue with no timeout while occupying one thread from a pool of ten. Every tracking customer therefore held a thread for the whole duration of the subscription, even while nothing was happening. Ten customers tracking their orders at once — entirely ordinary use — consumed every thread in the pool, after which the server answered nothing at all:

Load	Before	After
10 customers tracking	DEADLINE_EXCEEDED after 10.0s	OK in 0.00s
40 customers tracking	—	OK in 0.00s

This is worth recording because of *how* it hides. Every individual operation is correct. The system fails only when enough clients do a legitimate thing simultaneously, and the symptom — a timeout on an unrelated call — resembles a network problem rather than a design error. No test of a single client’s behaviour could have found it.

The repair has two parts. The queue is now waited on with a one-second timeout inside a loop that re-checks whether the client is still connected, so a thread is released promptly when a customer disconnects and is never parked indefinitely. And the pool was enlarged to 64 workers, because each tracking customer legitimately requires one thread while watching. After the change, forty simultaneous trackers leave the server responding immediately to `ListRestaurants` and `PlaceOrder`, and status updates continue to reach every subscriber in order.

The general lesson is that a thread-per-connection server has a client limit built into its thread pool, and that limit is invisible until it is crossed. It is worth knowing the number deliberately rather than discovering it.

5 Conclusions

The system implements all five required RPCs, enforces the order state machine, returns a distinct and meaningful gRPC status code for each of the six specified error conditions, and delivers status changes to subscribed customers through a server-streaming call without their asking.

The most instructive part of the exercise was not any of that. It was that a system passing every functional test still failed under ten concurrent users, for a reason that lived in the server’s thread pool rather than in any of its logic — and that finding it required measuring the system under load rather than reading the code.